BCA SEMESTER 5



Advanced DBMS

Subject Code: BCAMJ502

Course Credit: 02

Subject Faculty: Mr. Arjunsinh Vaghela

Parallel Database

What is Parallel Database?

A Parallel Database is a database system that uses multiple processors or multiple computers together to execute database operations simultaneously.

Instead of one CPU doing all work,

Many CPUs perform the work together.

This increases

- Speed
- Performance
- Throughput

Real Life Example

Imagine 100 students are waiting to pay fees.

Situation 1

Only one counter

100 Students

↓

Counter 1

Time = Very High

Situation 2

Five counters

Counter 1

Counter 2

Counter 3

Counter 4

Counter 5

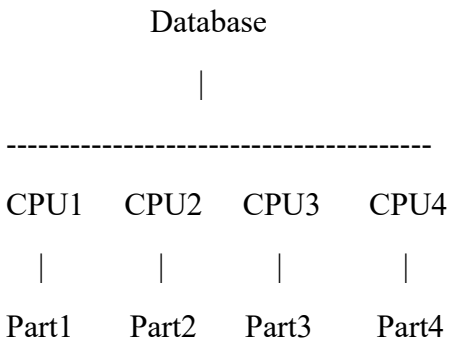
↓

Students divided among counters

Time = Much Less

Arjunsinh Vaghela

This is exactly how Parallel Database works.



Advantages

- High Speed
- Faster Query Execution
- Better Performance
- Supports Big Data
- Multiple users can access simultaneously
- Efficient Resource Utilization

Disadvantages

- Expensive Hardware
- Complex Design
- Synchronization Required
- Difficult Maintenance

Applications

- Banking
- Data Warehouse
- Scientific Research
- Weather Forecasting
- AI Systems

Arjunsinh Vaghela

Types of Parallel Databases

There are four major architectures.

1. Shared Memory Architecture

All CPUs share

Memory

Disk

2. Shared Disk Architecture

Each processor has its own memory.

All processors share only disk.

3. Shared Nothing Architecture

Every processor has

- Own CPU
- Own Memory
- Own Disk

4. Hybrid Architecture

Combination of

- Shared Memory
- Shared Disk
- Shared Nothing

Used in large cloud environments.

Parallel Query Processing

Suppose we want: **SELECT SUM(Salary) FROM Employee;**

Instead of calculating on one CPU,

CPU1 → Salary of Employees 1-1000

CPU2 → Salary of Employees 1001-2000

CPU3 → Salary of Employees 2001-3000

↓

Final Sum

Execution becomes much faster.

Arjunsinh Vaghela

Distributed Database

What is Distributed Database?

A Distributed Database is a collection of logically related databases that are physically stored at different geographical locations but appear to users as a single database.

Real Life Example

State Bank of India

Branches

- Ahmedabad
- Rajkot
- Delhi
- Mumbai

Each branch stores its own customer data.

All branches are connected.

Customer can withdraw money from any branch.

This is Distributed Database.

Advantages

- High Availability
- Faster Local Access
- Easy Expansion
- Better Reliability
- Fault Tolerance

Disadvantages

- Complex Management
- Network Dependency
- Security Issues
- Data Synchronization Problems

Arjunsinh Vaghela

Applications

- Banks
- Airlines
- Hospitals
- Universities
- Online Shopping

Types of Distributed Database

1. Homogeneous Distributed Database

    All sites use

        Same DBMS

        Same Operating System

    Example

        All branches use MySQL.

2. Heterogeneous Distributed Database

    Different sites use different DBMS.

    Example

        Ahmedabad → Oracle

        Mumbai → SQL Server

        Delhi → PostgreSQL

Distributed DBMS Architecture

User

↓

Application

↓

Distributed DBMS

↓

Site A

Arjunsinh Vaghela

Site B

Site C

The user never knows where data is stored.

Query Processing of Distributed Database

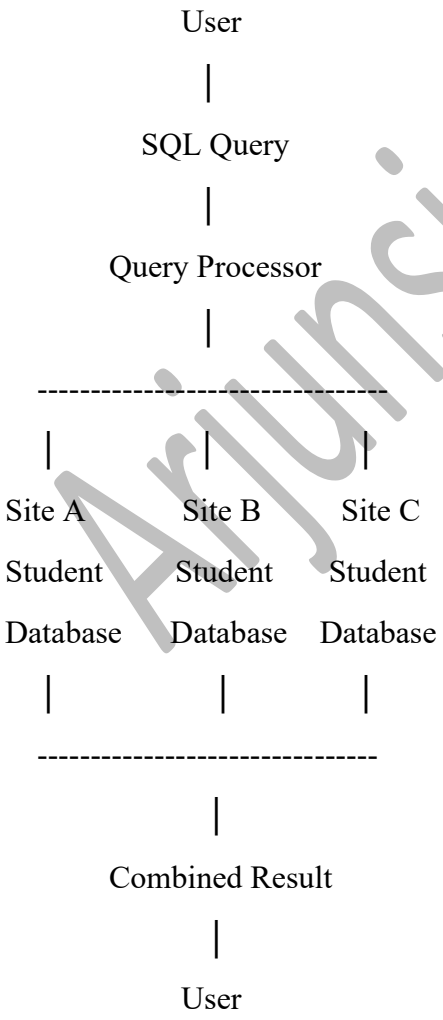
Suppose a university has campuses in:

- Ahmedabad
- Bhuj
- Surat

Each campus stores its own student records.

A teacher executes: **SELECT * FROM Student WHERE Marks > 80;**

The DBMS collects matching records from all campuses and combines them into one result.



Arjunsinh Vaghela

Distributed DBMS Architecture

1. Client-Server Architecture

Client

|

Database Server

Example: College ERP

Students access database using browser.

2. Peer-to-Peer Architecture

A Peer-to-Peer (P2P) Architecture is a distributed database architecture in which every computer (called a peer or node) has equal responsibility.

There is no central server.

Each computer can:

- Store data
- Send data
- Receive data
- Process queries

So, every node acts as both a client and a server.

Node1 ---- Node2

| |

Node3 ---- Node4

3. Multi-tier Architecture (Three-Tier Architecture)

A Multi-tier Architecture is a database architecture in which the application is divided into multiple layers (tiers). Each layer has a specific responsibility and communicates with the next layer.

The most common type is the Three-Tier Architecture:

- Presentation Layer (Client)

Arjunsinh Vaghela

- Application Layer (Business Logic)
- Database Layer (Database Server)

Difference between Parallel Database and Distributed Database

Parallel Database	Distributed Database
Same location	Different locations
Multiple CPUs	Multiple Sites
Speed improvement	Data sharing
High Performance	High Availability

ORDBMS: Structured Data Type

In a traditional Relational Database Management System (RDBMS), data is stored using **simple (primitive) data types** such as:

- INTEGER
- CHAR
- VARCHAR
- FLOAT
- DATE

These data types are suitable for storing simple values.

However, in modern applications such as:

- Banking
- Hospital Management
- E-commerce
- GIS (Maps)
- Student Management

we often need to store **complex information** that contains multiple related attributes.

To solve this problem, **ORDBMS introduces Structured Data Types (SDTs).**

Arjunsinh Vaghela

What is a Structured Data Type?

A **Structured Data Type (SDT)** is a user-defined data type that groups multiple related attributes into a single object.

Simple Definition

A Structured Data Type is a collection of different data fields stored together as one unit.

Real-Life Example

Imagine your **home address**.

Normally, your address consists of:

- House Number
- Street
- Area
- City
- State
- PIN Code

Instead of storing these separately everywhere, we create one object called **Address**.

Address

├── House No
├── Street
├── Area
├── City
├── State
└── PIN Code

Whenever we need an address, we simply use the **Address object**.

Why Structured Data Types are Required?

Suppose we are storing student information.

Without Structured Data Type

Arjunsinh Vaghela

Roll No	Name	House No	Street	City	State	PIN
101	Rahul	12	MG Road	Ahmedabad	Gujarat	380001

Every table must repeat all address columns.

Problems:

- More columns
- Data redundancy
- Difficult maintenance
- Repeated design

With Structured Data Type

Create one Address object.

Address

House No

Street

City

State

PIN

Now Student table becomes

Roll No	Name	Address
101	Rahul	Address Object

Much simpler.

Arjunsinh Vaghela

Structure of Structured Data Type

Address

House No

Street

City

State

PIN

The entire Address is treated as **one data type**.

Another Real-Life Example

Suppose a college stores faculty information.

Each faculty has

- Name
- Qualification
- Experience
- Department

Instead of writing these fields repeatedly,

Create

Faculty_Details

├── Qualification

Arjunsinh Vaghela

├── Experience

└── Department

Now every Faculty record stores one object.

Operations on ORDBMS Structured Data

Operation 1 – Creating Structured Data

Before using structured data, we first define its structure.

Example

```
CREATE TYPE Address_Type AS (
   HouseNo VARCHAR(10),
   Street VARCHAR(30),
   City VARCHAR(30),
   State VARCHAR(30),
   PIN VARCHAR(10)
);
```

Operation 2 – Creating a Table Using Structured Data

Once the type is created, it can be used as a column.

```
CREATE TABLE Student (
   RollNo INT,
   Name VARCHAR(50),
   Address Address_Type
);
```

Operation 3 – Inserting Structured Data

Suppose we want to insert:

Roll No : 101

Name : Rahul

Address:

House No : 12

Arjunsinh Vaghela

Street : MG Road

City : Ahmedabad

State : Gujarat

PIN : 380001

Conceptually:

```
INSERT INTO Student
VALUES (
   101,
   'Rahul',
   Address_Type(
      '12',
      'MG Road',
      'Ahmedabad',
      'Gujarat',
      '380001'
   )
);
```

```
Student
│
├── RollNo → 101
├── Name → Rahul
└── Address
     ├── HouseNo → 12
     ├── Street → MG Road
     ├── City → Ahmedabad
     ├── State → Gujarat
     └── PIN → 380001
```

Arjunsinh Vaghela

Operation 4 – Retrieving Complete Structured Data

We can retrieve the complete record using:

SELECT * FROM Student;

Operation 5 – Accessing Individual Attributes

This is one of the **most important operations**.

Suppose we only want the student's city.

We don't need the entire Address.

Conceptually:

SELECT Address.City FROM Student;

Operation 6 – Updating Structured Data

Suppose Rahul moves from Ahmedabad to Bhuj.

We need to update the City attribute.

Conceptually:

UPDATE Student

SET Address.City = 'Bhuj' WHERE RollNo = 101;

Updating Multiple Attributes

Suppose Rahul's complete address changes.

```
UPDATE Student
SET
    Address.Street = 'Station Road',
    Address.City = 'Bhuj',
    Address.State = 'Gujarat',
    Address.PIN = '370001'
WHERE RollNo = 101;
```

Arjunsinh Vaghela

Updating the Complete Structured Object

Sometimes we want to replace the complete Address.

Conceptually:

```
UPDATE Student
SET Address = Address_Type(
   '25',
   'Main Road',
   'Bhuj',
   'Gujarat',
   '370001'
)
WHERE RollNo = 101;
```

Operation 7 – Deleting Structured Data

There are two different meanings of "delete" here.

A. Delete the complete student record

```
DELETE FROM Student WHERE RollNo = 101;
```

Operation 8 – Searching Structured Data

We can use the attributes of a structured object in the WHERE clause.

Suppose we want students from Ahmedabad.

```
SELECT Name FROM Student WHERE Address.City = 'Ahmedabad';
```

Searching by PIN Code

```
SELECT Name, Address
FROM Student WHERE Address.PIN = '380001';
```

Searching by Multiple Attributes

Suppose we want students from Ahmedabad, Gujarat.

```
SELECT Name FROM Student
```

Arjunsinh Vaghela

WHERE Address.City = 'Ahmedabad'

AND Address.State = 'Gujarat';

Operation 9 – Sorting Structured Data

We can also sort records based on an attribute.

For example, sort students by city:

SELECT Name, Address.City

FROM Student

ORDER BY Address.City;

Operation 11 – Using Structured Data in Conditions

Suppose we want students whose PIN starts with 380.

Conceptually:

SELECT Name FROM Student WHERE Address.PIN LIKE '380%';

Operation 13 – Functions on Structured Data

We can apply functions to individual attributes.

For example:

SELECT UPPER(Address.City) FROM Student;

Object in ORDBMS

What is an Object?

An **object** is a real-world entity represented in a database as a combination of its data and, where supported, operations or methods.

Object = State + Behavior

An object generally has two important parts:

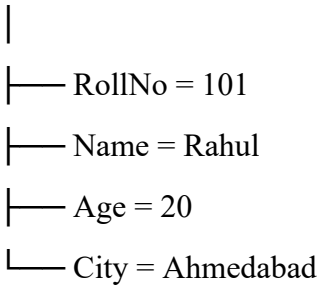
1. State

State means the data/properties of the object.

Example:

Arjunsinh Vaghela

Student

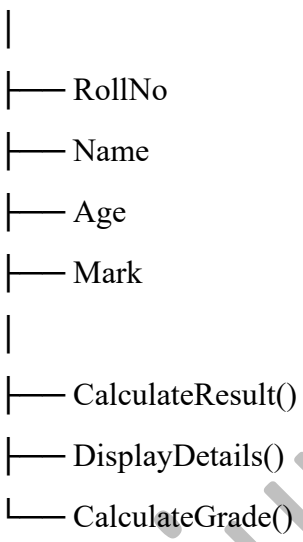


Behavior

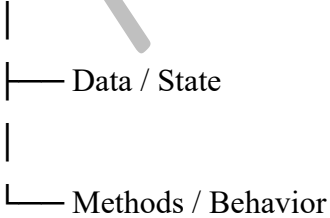
Behavior means the **operations/methods** that can be performed by the object.

Example:

Student



OBJECT



Arjunsinh Vaghela

Real-Life Example – Mobile Phone

Consider a mobile phone.

Properties

Brand = Samsung

Model = Galaxy

RAM = 8 GB

Storage = 256 GB

Behaviors

Call()

SendMessage()

TakePhoto()

PlayMusic()

So:

Mobile Phone Object

```
│
├── Data
│   ├── Brand
│   ├── Model
│   ├── RAM
│   └── Storage
│
└── Methods
    ├── Call()
    ├── SendMessage()
    └── TakePhoto()
```

This is the object-oriented way of representing a real-world entity.

Arjunsinh Vaghela

INHERITANCE: ORDBMS

11. What is Inheritance?

Inheritance is one of the most important concepts of Object-Oriented Programming and ORDBMS.

Inheritance is a mechanism in which a new class or type acquires the properties and behaviors of an existing class or type.

Simple Definition: Child gets the properties of Parent.

Advantages of Inheritance

1. Code/Data Reusability: Common attributes can be reused.
2. Avoids Duplication: No need to redefine common properties.
3. Better Organization: Classes/types are arranged logically.
4. Easy Maintenance: Changes to common properties can be managed at the parent level.

Types of Inheritance

1 Single Inheritance

One child inherits from one parent.

Person

|

↓

Student

---

2 Multilevel Inheritance

Inheritance occurs across multiple levels.

Person

↓

Employee

↓

Manager

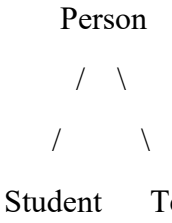
Arjunsinh Vaghela

Manager inherits from Employee, and Employee inherits from Person.

---

3 Hierarchical Inheritance

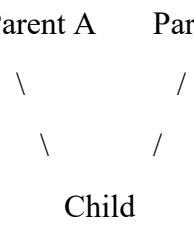
One parent has multiple children.



---

4 Multiple Inheritance

One child inherits from multiple parents.



Example of User-Defined Types

Conceptually:

```
CREATE TYPE Person_Type AS OBJECT (
   Name VARCHAR(50),
   Age NUMBER
);
```

Then a child type may extend the parent type in systems that support type inheritance.

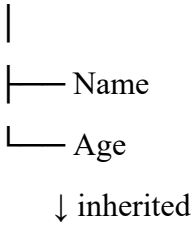
For example, in an Oracle-style object model:

```
CREATE TYPE Student_Type UNDER Person_Type (
   RollNo NUMBER
);
```

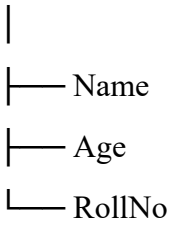
Arjunsinh Vaghela

Conceptually:

Person_Type



Student_Type



This syntax is specifically associated with Oracle's object-relational features; students should not assume that every ORDBMS uses exactly the same syntax.

Transaction Processing Concepts

A database is continuously used for operations such as:

- Insert data
- Update data
- Delete data
- Transfer money
- Book tickets
- Place orders
- Update marks
- Register students

Many of these operations involve **multiple database actions that must be treated as one logical unit**.

This logical unit is called a **Transaction**.

What is a Transaction?

A **transaction** is a group of database operations that together perform **one complete task**.

Arjunsinh Vaghela

Example: Bank Transfer

Suppose Rahul transfers ₹5,000 to Amit.

The transaction involves:

Transaction

├── Deduct ₹5,000 from Rahul

└── Add ₹5,000 to Amit

Both operations must be completed successfully.

If Rahul's money is deducted but Amit does not receive it, the database becomes incorrect.

What is Transaction Processing?

Transaction Processing is the process of executing database transactions while maintaining the correctness and consistency of the database.

In simple words:

It ensures that a database operation is completed properly or its incomplete changes are cancelled.

Why Do We Need Transaction Processing?

Imagine a banking system without proper transaction processing.

Rahul has: Balance = ₹20,000

He transfers: ₹5,000 to Amit.

The system performs: Rahul: ₹20,000 → ₹15,000

But suddenly the computer crashes.

Amit's account remains: ₹10,000 instead of: ₹15,000

Now ₹5,000 has disappeared.

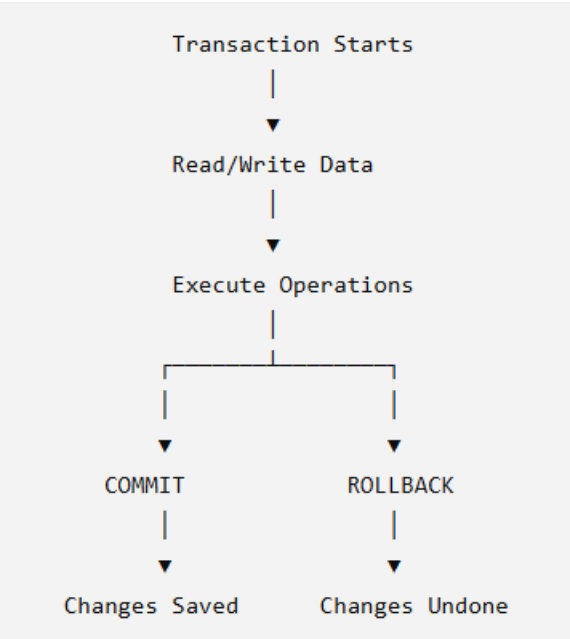
This is an example of an **incomplete transaction**.

Transaction processing prevents such situations.

Arjunsinh Vaghela

Basic Transaction Processing Model

A transaction generally follows this sequence:



Main Operations of a Transaction

A transaction commonly performs two fundamental database operations:

1. Read

Read data from the database.

Represented as: **Read(X)**

It means: Read the value of data item X.

2. Write

Write or modify data in the database.

Represented as: **Write(X)**

It means: Change the value of data item X.

Transaction States

A transaction does not immediately become successful.

It passes through different states.

The major transaction states are:

Arjunsinh Vaghela

1. **Active:** When a transaction starts executing, it is in the **Active** state.
2. **Partially Committed:** When the transaction has completed its final operation but the changes have not yet been permanently stored, it enters the **Partially Committed** state.
3. **Committed**: If all operations are successful, the transaction executes:
4. **Failed**: If a transaction cannot continue because of an error or system failure, it enters the **Failed** state.
5. **Aborted**: After a transaction fails, its changes may need to be undone. This process is called **Rollback**.
6. **Terminated**: After a transaction is committed or aborted and all required cleanup is completed, the transaction terminates.

Commit

COMMIT permanently saves all changes made by a transaction.

Example:

```
UPDATE Account

SET Balance = Balance - 5000

WHERE AccountNo = 101;


COMMIT;
```

After COMMIT, the changes are saved permanently.

Rollback

ROLLBACK cancels the changes made by the current transaction and restores the database to an appropriate previous state.

Example:

```
UPDATE Account

SET Balance = Balance - 5000

WHERE AccountNo = 101;


ROLLBACK;
```

The update is undone.

Arjunsinh Vaghela

COMMIT vs ROLLBACK

COMMIT	ROLLBACK
Saves changes	Undoes changes
Makes changes permanent	Cancels changes
Successful transaction	Failed/cancelled transaction
Database moves to new valid state	Database returns to previous appropriate state

Transaction and System Concepts

What does "System Concepts" mean?

In this topic, **System Concepts** means understanding:

How a DBMS manages, controls, and executes a transaction safely.

In simple words:

Transaction tells us WHAT task has to be performed.

System concepts explain HOW the DBMS manages that task.

Example

A customer wants to transfer ₹5,000 from Account A to Account B.

WHAT?

Transfer ₹5,000

↓

TRANSACTION

But the DBMS must also decide:

HOW?

Manage the transaction

↓

Handle other transactions

↓

Handle failures

↓

Update the database safely

This is the idea behind **Transaction and System Concepts**.

Arjunsinh Vaghela

Desirable Properties of a Transaction

The most important desirable properties of a transaction are known as **ACID properties**.

ACID = Atomicity + Consistency + Isolation + Durability

These properties ensure that database transactions are **correct, reliable, and safe**.

Atomicity

Atomicity means a transaction is treated as one indivisible unit: either all required operations are completed, or the transaction is undone.

In simple words:

All or Nothing

Example

Transfer ₹5,000:

T1

├── Debit A ₹5,000 ✓

└── Credit B ₹5,000 ✓

↓

COMMIT

If the second operation fails:

T1

├── Debit A ₹5,000 ✓

└── Credit B ₹5,000 ✗

↓

ROLLBACK

The debit should also be undone.

Consistency

Consistency means a transaction must take the database from one valid state to another valid state.

The transaction must follow all database rules and constraints.

Example

Arjunsinh Vaghela

Suppose:

Account A = ₹10,000

Account B = ₹5,000

Total:

₹15,000

Transfer ₹2,000 from A to B:

A = ₹8,000

B = ₹7,000

Total:

₹15,000

The database remains logically correct.

Example 2:

If a college database has a rule:

Student marks must be between 0 and 100.

A transaction should not leave:

Marks = 150

because that violates the database rule.

Isolation

Isolation means that concurrently executing transactions should not improperly interfere with each other.

In a multi-user database, many transactions can run at the same time.

Example:

T1 → Withdraw ₹5,000

T2 → Check Account Balance

T3 → Deposit ₹2,000

These transactions may execute concurrently.

The DBMS should control their interaction so that one transaction does not see inappropriate intermediate results from another.

Arjunsinh Vaghela

Durability

Durability means that once a transaction has been successfully committed, its changes must remain saved even if a system failure occurs afterward.

Example

Suppose:

Account A = ₹20,000

Withdraw ₹5,000

A = ₹15,000

COMMIT ✓

Now the system crashes:

SYSTEM CRASH

After the system recovers:

Account A = ₹15,000

The committed change must not disappear.

ACID Properties

Property	Meaning	Easy Keyword
Atomicity	All operations happen or none	**All or Nothing**
Consistency	Database remains valid	**Correctness**
Isolation	Transactions don't improperly interfere	**Separation**
Durability	Committed changes remain saved	**Permanent**

Why Are ACID Properties Important?

ACID properties provide:

- **Reliable transactions**
- **Correct database results**
- **Data integrity**
- **Protection from transaction failures**
- **Safe multi-user access**
- **Persistence of committed data**

Arjunsinh Vaghela

They are especially important in:

- Banking systems
- Railway reservation
- E-commerce
- Hospital systems
- University databases
- Payment systems

Schedules and Recoverability

This topic is very important because in a **multi-user database**, many transactions may execute at the same time. The DBMS must decide **in which order their operations are executed** and must ensure that the database can be safely recovered if a transaction fails.

What is a Schedule?

A **schedule** is the order in which the operations of multiple transactions are executed.

Why are Schedules Needed?

In a multi-user DBMS, transactions often execute concurrently.

For example:

T1 → Update Student Fees

T2 → Update Student Result

T3 → View Student Details

The DBMS needs to determine how their operations are arranged.

A schedule helps the DBMS:

- Organize transaction operations
- Execute transactions concurrently
- Maintain database consistency
- Analyze whether execution is safe

Arjunsinh Vaghela

What is Recoverability?

Now suppose a transaction fails.

We need to know whether the database can still be recovered safely.

Definition

A **recoverable schedule** is a schedule in which a transaction commits only after the transactions from which it has read data have committed.

Simple Meaning

A transaction should not permanently commit based on data produced by another transaction that has not yet committed.

Non-Recoverable Schedule

Consider:

T1: Write(A)

T2: Read(A)

T2: COMMIT

T1: ROLLBACK

Problem:

T2 has already committed using data written by T1.

Then T1 fails and rolls back.

T1 → Write(A)

↓

T2 → Read(A)

↓

T2 → COMMIT ✓

↓

T1 → ROLLBACK ✗

Now T2 contains a result based on data that has been undone.

Therefore, the schedule is **non-recoverable**.

Arjunsinh Vaghela

Serializability of Schedules

What is Serializability?

Serializability is the property of a non-serial schedule that produces the same final result as some serial schedule.

Simple Meaning

A transaction schedule may be executed **concurrently**, but its final result should be the same as if the transactions had been executed **one by one**.

Serial vs Serializable

These two terms are different.

Serial Schedule

Transactions actually execute one after another.

T1 → T1 → T1 → T2 → T2 → T2

Serializable Schedule

Transactions may be interleaved, but the final result is equivalent to some serial schedule.

T1 → T2 → T1 → T2

This may still be correct if its result is equivalent to:

T1 → T2

or

T2 → T1

Simple Example

Consider two transactions.

T1

Read(A)

Write(A)

T2

Read(B)

Write(B)

Suppose the schedule is:

Arjunsinh Vaghela

T1: Read(A)

T1: Write(A)

T2: Read(B)

T2: Write(B)

This is serial:

T1 → T2

Therefore, it is serializable.

Transaction Support in SQL

SQL provides special commands to **start, control, save, and cancel transactions**. These commands are called **Transaction Control Commands**.

What is Transaction Support in SQL?

Transaction support in SQL means the facilities provided by SQL to manage a group of database operations as one transaction.

The important commands are:

BEGIN / START TRANSACTION

↓

SQL Operations

↓

COMMIT / ROLLBACK

The exact BEGIN syntax varies between DBMS products, but the concepts are common.

SAVEPOINT

Sometimes we do not want to cancel the **entire transaction**.

We may want to go back only to a particular point.

For this, SQL provides a **SAVEPOINT**.

Definition

A SAVEPOINT creates a point within a transaction to which the transaction can later be rolled back.

Arjunsinh Vaghela

Example

```
UPDATE Account
SET Balance = Balance - 5000
WHERE AccountNo = 101;

SAVEPOINT S1;

UPDATE Account
SET Balance = Balance + 5000
WHERE AccountNo = 102;

ROLLBACK TO S1;
```

The transaction is rolled back to S1, rather than necessarily cancelling everything before it.

Easy Meaning

SAVEPOINT = Temporary checkpoint

Transaction Control Language (TCL)

Commands such as COMMIT, ROLLBACK, and SAVEPOINT are commonly discussed under **Transaction Control Language (TCL)**.

Main TCL Commands

COMMIT

ROLLBACK

SAVEPOINT

Some DBMSs also provide additional transaction-related commands.

Arjunsinh Vaghela

Recovery Techniques

What is Recovery?

A database may fail while transactions are running due to:

- Power failure
- System crash

- Hardware failure
- Software error
- Transaction failure

Database Recovery is the process of restoring the database to a **correct and consistent state after a failure**.

Simple Definition

Recovery is the process of bringing the database back to a correct state after a failure.

Main Recovery Techniques

The important recovery techniques for this syllabus are:

1. **Log-Based Recovery**
2. **Checkpointing**
3. **Deferred Update**
4. **Immediate Update**
5. **Shadow Paging**

Log-Based Recovery

Definition

Log-based recovery records transaction activities in a log so that the database can be recovered after a failure.

A log may contain information such as:

<T1, Start>

<T1, A, 10000, 8000>

<T1, B, 5000, 7000>

<T1, Commit>

This tells the DBMS:

- Which transaction started
- What data was changed
- Old value
- New value
- Whether it committed

Arjunsinh Vaghela

Checkpointing

A **checkpoint** is a point during execution at which the DBMS records a known recovery position, reducing the amount of work required during recovery.

Without checkpoints, after a crash the DBMS might need to examine a very large portion of the log.

With checkpoints:

Start

↓

Transactions

↓

CHECKPOINT

↓

More Transactions

↓

System Crash

Recovery can focus on the relevant part of the log.

Deferred Update

In **Deferred Update**, changes made by a transaction are not written to the database until the transaction commits.

Transaction

↓

Changes

↓

Log / Temporary Storage

↓

COMMIT

↓

Database Update

Advantage

If the transaction fails before commit, its uncommitted database changes do not need to be undone from the database.

Arjunsinh Vaghela

Immediate Update

In **Immediate Update**, changes may be written to the database before the transaction commits.

Transaction

↓

Update Database

↓

More Operations

↓

COMMIT

If the transaction fails, the DBMS may need to **UNDO** its changes.

Therefore, logging is very important.

Shadow Paging

Shadow paging is a recovery technique in which the system maintains an original page table and a modified copy, allowing it to preserve the previous database state until a transaction commits.

Simple idea:

Original Pages

↓

Shadow Page Table

Modified Pages

↓

New Page Table

If the transaction succeeds:

New Page Table

↓

Become Current

If the transaction fails:

Discard New Pages

↓

Arjunsinh Vaghela

Use Original Pages

Arjunsinh Vaghela